

План доработки приложения «Швейное производство»

Анализ архитектуры, безопасности, качества кода и UX

с приоритизированным списком задач

Стек: Next.js 16 + Prisma 6 + SQLite + Bun

Дата: 31 мая 2026

Содержание

1. Общая оценка приложения	4
2. Критические задачи (блокеры)	4
2.1. Починка аутентификации: верификация подписи JWT	5
2.2. Защита эндпоинтов seed и print	5
2.3. Починка сломанного эндпоинта /api/auth/session	6
2.4. Добавление транзакций в критические операции	6
3. Задачи высокого приоритета	7
3.1. Устранение TypeScript-антипаттернов	7
3.2. Добавление базового тестирования	8
3.3. Унификация обработки ошибок	9
3.4. Синхронизация статусов в Zod-схемах и Prisma	9
3.5. Валидация реквизитов (ИНН/КПП/БИК)	10
4. Функциональные пробелы	10
4.1. Реализация ролей Технолог, Расройщик, Продавец	10
4.2. Смена пароля пользователем	11
4.3. Пагинация и фильтрация списков	11
4.4. Дашборд супервизора	11
4.5. Подтверждение деструктивных действий	12

4.6. Загрузка изображений продуктов	12
5. Улучшения UX и качества	13
5.1. Мобильная адаптация	13
5.2. Уведомления о статусных изменениях	13
5.3. Экспорт данных	13
5.4. Очистка неиспользуемых зависимостей	14
5.5. Удаление модели Task (legacy)	14
6. Оптимизация и масштабируемость	15
6.1. Оптимизация N+1 запросов	15
6.2. Стратегия миграции с SQLite	15
6.3. Добавление индексов в Prisma-схему	16
7. Сводная таблица задач	16
8. Рекомендуемый порядок реализации	17

1. Общая оценка приложения

Приложение представляет собой полнофункциональную платформу автоматизации швейного производства, охватывающую 10 взаимосвязанных модулей: планирование, раскрой, пошив, контроль качества, утюжка, распределение, упаковка, материалы, финансы и CRM. Доменная модель проработана детально, включены такие сложные механизмы, как поддержка комплектов (кит-продуктов), автосписание материалов при раскрое, авторасчёт норм, поштучный трекинг статусов с временными метками и расчёт зарплат. Это серьёзная и продуманная система, решающая реальные бизнес-задачи швейного предприятия.

Ниже представлена сводная оценка по ключевым направлениям:

Категория	Оценка	Комментарий
Функциональность	7/10	Основной поток работает; 3 роли-заглушки; финансы базовые
Качество кода	5/10	Хорошая структура, но апу повсюду; нет тестов; дублирование
Безопасность	2/10	Подделка токена тривиальна; seed/print без авторизации
Целостность данных	4/10	Нет транзакций; race conditions; ручные каскадные удаления
UX	6/10	Функционально, но нет отмены, мобильной адаптации, уведомлений
Производительность	5/10	Нормально для малого масштаба; нет пагинации; SQLite
Тестируемость	0/10	Нет ни одного теста; не настроен фреймворк

2. Критические задачи (блокеры)

Эти задачи требуют немедленного решения, поскольку они делают приложение уязвимым для атак и могут привести к потере или повреждению данных. Без их решения развивать функционал дальше

нецелесообразно — любая продакшн-эксплуатация невозможна.

2.1. Починка аутентификации: верификация подписи JWT

⚠ **Приоритет: P0 (блокер)**

Текущая реализация токена в файле `src/lib/auth.ts` создаёт JWT-подобную структуру из трёх частей (`header.payload.signature`), однако подпись представляет собой простую конкатенацию base64-строк, а функция `verifyToken()` только декодирует `payload` и никогда не проверяет подпись. Это означает, что любой злоумышленник может сформировать токен с произвольным `userId` и `role` (например, `supervisor`), и сервер примет его как легитимный.

Что сделать:

- Заменить кастомный JWT на библиотеку `jsonwebtoken` или `jose`
- Использовать HMAC-SHA256 с секретом из `JWT_SECRET` для подписи
- В `verifyToken()` проверять подпись перед декодированием `payload`
- Убрать хранение токена в `localStorage` (оставить только `httpOnly cookie`)
- Сгенерировать надёжный `JWT_SECRET` вместо дефолтного `dev-secret-change-me`

Файлы:

- `src/lib/auth.ts` — основная реализация
- `src/components/auth-provider.tsx` — убрать `localStorage`, оставить `cookie`

2.2. Защита эндпоинтов `seed` и `print`

⚠ **Приоритет: P0 (блокер)**

Эндпоинт `GET /api/seed` полностью удаляет все данные из базы и создаёт пользователей с захардкоженными паролями (`123456`, `admin`) без какой-либо авторизации. Любой, кто знает URL, может уничтожить все данные одним запросом. Эндпоинт `GET /api/print` генерирует печатные формы (наряды на раскрой, пошив, упаковочные листы) также без авторизации, что даёт доступ к коммерческой информации о производстве.

Что сделать:

- Добавить `withAuth(["supervisor"])` на оба эндпоинта
- В `seed`-эндпоинте дополнительно проверять `NODE_ENV=development`
- Заменить захардкоженные пароли на генерацию случайных
- Рассмотреть полное удаление `seed`-эндпоинта из продакшн-сборки

Файлы:

- `src/app/api/seed/route.ts`
- `src/app/api/print/route.ts`

2.3. Починка сломанного эндпоинта `/api/auth/session`

⚠️ Приоритет: P0 (блокер)

Эндпоинт `GET /api/auth/session` читает cookie с именем `"session"`, тогда как логин-эндпоинт устанавливает cookie с именем `"token"`. Кроме того, `session` использует метод декодирования, отличный от `verifyToken()`. В результате эндпоинт всегда возвращает ошибку, и сессионная проверка не работает. Если какой-либо компонент пытается использовать этот эндпоинт для проверки авторизации, пользователь всегда будет считаться неавторизованным.

Что сделать:

- Изменить чтение cookie с `"session"` на `"token"`
- Использовать `verifyToken()` вместо кастомного декодирования
- Унифицировать все эндпоинты авторизации для использования одной функции проверки

Файлы:

- `src/app/api/auth/session/route.ts`
- `src/app/api/auth/login/route.ts` — сверить имя cookie

2.4. Добавление транзакций в критические операции

⚠️ Приоритет: P0 (блокер)

Множественные операции записи в базу данных выполняются без транзакций. Если одна из операций в середине цепочки завершается ошибкой, предыдущие изменения остаются в базе, приводя к неконсистентному состоянию. Это особенно критично для операций одобрения плана (создание наряда на раскрой + обновление статуса), списания материалов, создания задач на пошив и удаления записей с каскадными зависимостями.

Ключевые операции без транзакций:

Операция	Файл	Риск при сбое
Одобрение плана	api/plans/[id]	План approved, но наряд на раскрой не создан
Списание материалов	api/cutting-plans/[id]	Статус cut, но остатки не обновлены
Создание задачи на пошив	api/sewing-tasks	Задача создана, а позиции — нет
Удаление плана продаж	api/seller-plans/[id]	Часть позиций удалена, часть — нет
Обновление остатков	api/material-entries	read-then-update race condition

Что сделать:

- Обернуть все мульти-записи в `db.$transaction()`
- Для read-then-update — использовать `$transaction` с `interactive transactions`
- Добавить обработку ошибок транзакций с откатом

3. Задачи высокого приоритета

Эти задачи не блокируют работу приложения, но значительно влияют на его надёжность, поддерживаемость и безопасность данных. Их решение необходимо до выхода в продакшн.

3.1. Устранение TypeScript-антипаттернов

► Приоритет: P1

В конфигурации TypeScript установлено `noImplicitAny: false` и `ignoreBuildErrors: true` в `next.config.ts`. Это позволяет компилировать код с неявными типами `any` и скрывает ошибки типизации при сборке. В результате в коде повсеместно используются типы `any` (`task: any`, `item: any`, `error: any`), что полностью лишает TypeScript его основной ценности — статической проверки типов. Ошибки в типах обнаруживаются только в рантайме.

Что сделать:

- Установить `noImplicitAny: true` в `tsconfig.json`
- Убрать `ignoreBuildErrors: true` из `next.config.ts`
- Добавить `reactStrictMode: true`
- Заменить все `any` на конкретные типы (Prisma генерирует их автоматически)
- Для `error` — использовать тип `unknown` с `type guard`
- Выполнять поэтапно: сначала включить `strict`, затем исправлять ошибки

3.2. Добавление базового тестирования

► Приоритет: P1

В приложении нет ни одного теста. Не настроен тестовый фреймворк, нет конфигурации CI/CD. Любое изменение может сломать существующий функционал без какого-либо уведомления. Это особенно опасно учитывая сложную доменную логику с множеством статусных переходов и взаимозависимых операций (списание материалов, расчёт зарплат, формирование документов).

Что сделать:

- Настроить Vitest как тестовый фреймворк (быстрый, совместим с Bun)
- Написать unit-тесты для `auth.ts` (`signToken`, `verifyToken`, `withAuth`)
- Написать unit-тесты для критических API-маршрутов (`plans`, `sewing-tasks`)
- Настроить in-memory SQLite для тестовой БД
- Добавить npm-скрипт `test` и `pre-commit` хук

3.3. Унификация обработки ошибок

► Приоритет: P1

Каждый API-маршрут обрабатывает ошибки Prisma (P2002, P2025) inline без общей утилиты. Формат ответа не унифицирован: одни эндпоинты возвращают { error }, другие { message }. Клиентский api-client.ts проверяет оба поля. Ошибки логируются через console.error без структурирования и без возможности трассировки. Нет request ID для связывания клиентских и серверных логов.

Что сделать:

- Создать утилиту handleApiError(error) с маппингом Prisma-кодов
- Унифицировать формат ответа: { error: string, code?: string }
- Обновить api-client.ts для единого формата
- Добавить генерацию request ID в proxy.ts/middleware
- Рассмотреть структурированный логгер (pino/winston) вместо console.error

3.4. Синхронизация статусов в Zod-схемах и Prisma

► Приоритет: P1

Обнаружено расхождение между статусами в Zod-схеме и Prisma-модели для CuttingLeftover. Zod определяет статусы: available, in_work, completed. Prisma-модель определяет: free, assigned, partially_sewn, fully_sewn. Это означает, что валидация на входе пропускает значения, которые не существуют в БД, и наоборот. Подобные расхождения могут существовать и для других моделей. Кроме того, логика статусных переходов дублируется между sewing-tasks/[id] и ironing/route.ts.

Что сделать:

- Провести аудит всех Zod-схем на соответствие Prisma-перечислениям
- Создать единый источник истины для статусных переходов (state machine)
- Вынести duplicated статус-логику в shared-модуль
- Добавить типы PrismaEnum для автоматической синхронизации

3.5. Валидация реквизитов (ИНН/КПП/БИК)

► Приоритет: P1

Поля ИНН, КПП, БИК и расчётный счёт хранятся как простой текст без валидации. Российский ИНН имеет контрольную цифру, которая позволяет обнаружить опечатку. КПП состоит из 9 цифр с определённой структурой. БИК — 9 цифр, начинающихся с 04. Расчётный счёт — 20 цифр с контрольной суммой, зависящей от БИК. Без валидации в базу попадают некорректные реквизиты, что делает невозможной генерацию корректных договоров, счетов и УПД, а также оплату по QR-коду.

Что сделать:

- Добавить функцию `validateINN()` с проверкой контрольной цифры
- Добавить валидацию КПП (9 цифр, структура)
- Добавить валидацию БИК (9 цифр, начинается с 04)
- Добавить валидацию расчётного счёта с контрольной суммой по БИК
- Интегрировать валидацию в Zod-схемы (`refine/superRefine`)
- Показывать понятные ошибки в UI при некорректных реквизитах

4. Функциональные пробелы

Эти задачи закрывают функциональные пробелы в текущей реализации. Они важны для полноценной работы пользователей, но не являются блокерами для базовой эксплуатации.

4.1. Реализация ролей Технолог, Раскройщик, Продавец

Три роли из шести отображают экран-заглушку «В разработке». Роль «Технолог» должна управлять технологическими картами изделий, нормы расхода материалов и последовательность операций. Роль «Раскройщик» уже имеет API (`cutting-plans`), но не имеет собственного UI. Роль «Продавец» перенаправляет на «Производство» вместо отображения интерфейса отгрузки и работы с клиентами.

Что сделать:

- Раскройщик: создать `dedicated UI` с существующим API `cutting-plans`

- Технолог: спроектировать и реализовать модуль технологических карт
- Продавец: создать интерфейс отгрузки, связи с CRM и заказчиками

4.2. Смена пароля пользователем

В системе отсутствует механизм смены пароля. Пользователи, созданные через seed-эндпоинт, получают захардкоженные пароли (123456, admin), которые невозможно сменить. Это означает, что все пользователи работают с известными паролями, что является серьёзной уязвимостью. Необходим как минимум функционал смены собственного пароля и сброса пароля администратором.

Что сделать:

- Создать API PATCH /api/auth/change-password
- Добавить UI смены пароля в профиль пользователя
- Добавить API PATCH /api/users/[id]/reset-password для супервизора
- Принудить смену пароля при первом входе

4.3. Пагинация и фильтрация списков

Все API-эндпоинты возвращают полные наборы данных без пагинации. При росте объёма данных (тысячи планов, десятки тысяч задач на пошив) это приведёт к медленной загрузке интерфейса, большому потреблению памяти на сервере и долгому рендерингу на клиенте. Только plans и sewing-tasks поддерживают query-параметры для фильтрации.

Что сделать:

- Добавить пагинацию (cursor-based для Prisma) во все list-эндпоинты
- Добавить параметры ?page=1&limit;=50&search;=&status;=
- Обновить UI-компоненты для работы с пагинацией
- Добавить индексы в Prisma-схему для часто фильтруемых полей

4.4. Дашборд супервизора

Эндпоинт /api/stats существует и возвращает базовую статистику, однако в интерфейсе нет dedicated вкладки дашборда для супервизора. Супервизор видит сразу список планов, а не сводку состояния

производства. Для эффективного управления необходим обзорный экранс ключевыми метриками: количество заказов в работе, просроченныезадачи, загрузка швей, остатки материалов и финансовые показатели.

Что сделать:

- Создать вкладку «Дашборд» как стартовый экран супервизора
- Показать KPI-карточки: заказы в работе, просрочки, загрузка, финансы
- Добавить графики трендов (выполнение по дням/неделям)
- Добавить алерты для критических ситуаций

4.5. Подтверждение деструктивных действий

Удаление планов, сотрудников, коробок и других сущностей выполняется без подтверждения. Случайный клик по кнопке удаления приводит к необратимой потере данных. Нет механизма отмены (undo), нет soft-delete, нет аудита изменений. Для производственной системы это критический UX-пробел, который может привести к потере важных данных.

Что сделать:

- Добавить диалог подтверждения на все delete-операции
- Реализовать soft-delete (поле deletedAt в моделях)
- Добавить логирование изменений (audit log)
- Реализовать корзину с восстановлением удалённых записей

4.6. Загрузка изображений продуктов

В модели Product существует поле imageUrl, однако не реализован механизм загрузки изображений. Для швейного производства визуальное представление изделия критически важно — без фотографий или эскизов сложно идентифицировать продукт при планировании, раскрое и пошиве. В package.json указана зависимость sharp, но она не используется.

Что сделать:

- Создать API POST /api/upload для загрузки изображений

- Использовать `sharp` для оптимизации и создания миниатюр
- Добавить UI загрузки в карточку продукта
- Хранить файлы локально с возможностью миграции на S3

5. Улучшения UX и качества

5.1. Мобильная адаптация

Текущий интерфейс не адаптирован для мобильных устройств. Вертикальный сайдбар и сеточные макеты плохо работают на экранах шириной менее 768px. Для производственной среды, где сотрудники могут использовать планшеты или телефоны для доступа к системе (например, Швея отмечает выполненную задачу со смартфона), мобильная адаптация необходима.

Что сделать:

- Добавить hamburger-меню для мобильных экранов
- Адаптировать табличные представления для узких экранов
- Протестировать ключевые сценарии на 375px и 768px

5.2. Уведомления о статусных изменениях

Пользователи не получают уведомлений об изменениях статусов. Швея не узнаёт о новой задаче, контролёр ОТК не видит поступления на проверку, супервизор не информирован о завершении заказа. Все изменения обнаруживаются только при ручном обновлении страницы. React Query автоматически обновляет данные с `staleTime=60s`, но это не уведомляет пользователя.

Что сделать:

- Добавить WebSocket/SSE для real-time обновлений
- Реализовать toast-уведомления при статусных переходах
- Добавить счётчик непросмотренных событий в сайдбаре

5.3. Экспорт данных

Отсутствует функционал экспорта данных в форматы Excel/CSV. Для производственного учёта возможность выгрузки отчётов по зарплатам, выполненным заказам, остаткам материалов и финансовым показателям является обязательной. Бухгалтерия и менеджмент работают с Excel-файлами, и ручной перенос данных из веб-интерфейса неэффективен.

Что сделать:

- Добавить API GET `/api/export?format=xlsx&type=salary`
- Использовать библиотеку `xlsx/exceljs` для генерации
- Добавить кнопки экспорта в ключевые табличные представления

5.4. Очистка неиспользуемых зависимостей

В `package.json` числится 80+ зависимостей, из которых значительная часть не используется в исходном коде: `next-auth`, `next-intl`, `framer-motion`, `qrcode`, `docx`, `zustand`, `react-markdown`, `react-syntax-highlighter`, `sharp`, `uuid`, `z-ai-web-dev-sdk`. Каждая неиспользуемая зависимость увеличивает размер `node_modules`, замедляет установку и сборку, а также создаёт потенциальные уязвимости безопасности.

Что сделать:

- Провести аудит с помощью `depcheck`
- Удалить неиспользуемые зависимости
- Заместить `qrcode` (уже генерирует QR) — проверить использование
- Оставить `sharp` если планируется загрузка изображений

5.5. Удаление модели Task (legacy)

В Prisma-схеме осталась модель `Task` от старой системы задач. `Seed`-эндпоинт создаёт записи в этой таблице, но реальная система использует `SewingTask/SewingTaskItem`. Наличие устаревшей модели загрязняет схему, создаёт путаницу и может приводить к ошибкам при разработке. API `/api/tasks` также является legacy и должен быть удалён после миграции.

Что сделать:

- Убедиться, что Task нигде не используется в UI
- Удалить модель Task из schema.prisma
- Удалить /api/tasks route
- Удалить создание Task из seed-эндпоинта
- Применить миграцию

6. Оптимизация и масштабируемость

6.1. Оптимизация N+1 запросов

Ряд API-эндпоинтов выполняет запросы к базе данных в цикле, что создаёт проблему N+1. Например, при обновлении наряда на раскрой каждая позиция обновляется отдельным запросом. Расчёт доступных позиций для распределения загружает все задачи на пошив и все планы продаж полностью. С ростом объёма данных это приведёт к значительному замедлению.

Что сделать:

- Заменить циклы с individual DB calls на batch operations
- Использовать Prisma updateMany/createMany для массовых операций
- Оптимизировать запрос available-items через агрегацию на уровне БД

6.2. Стратегия миграции с SQLite

SQLite поддерживает только одного писателя одновременно. При одновременных запросах на запись (например, несколько швей одновременно отмечают выполнение задач) база данных блокируется, и запросы ставятся в очередь. Для одного-двух пользователей это не проблема, но при росте количества одновременных пользователей система станет медленной и начнёт выдавать ошибки SQLITE_BUSY. Кроме того, absolute path в DATABASE_URL (/home/z/my-project/db/custom.db) может не работать при деплое с output: standalone.

Что сделать:

- На ближайшем этапе: добавить WAL-режим и настройки `busy_timeout`
- Среднесрочно: подготовить миграцию на PostgreSQL
- Использовать относительный путь для `DATABASE_URL`

6.3. Добавление индексов в Prisma-схему

В текущей схеме Prisma отсутствуют явные индексы для часто используемых полей фильтрации и сортировки: статус задач, дата создания, идентификатор сотрудника, идентификатор плана. По мере роста объёма данных запросы без индексов будут выполняться всё медленнее. Prisma позволяет объявлять индексы декларативно, и их добавление не требует изменения логики.

Что сделать:

- Добавить `@@index` на поля `status` во всех моделях
- Добавить `@@index` на `createdAt` для сортировки по дате
- Добавить `@@index` на `employeeId`, `planId` для фильтрации
- Создать миграцию и проверить скорость запросов

7. Сводная таблица задач

#	Приоритет	Задача	Затронутые модули
2.1	P0	Верификация подписи JWT	auth.ts
2.2	P0	Защита seed и print	api/seed, api/print
2.3	P0	Починка /api/auth/session	api/auth/session
2.4	P0	Добавление транзакций	plans, cutting, sewing
3.1	P1	Устранение any и ignoreBuildErrors	tsconfig, next.config
3.2	P1	Базовое тестирование (Vitest)	auth, API routes

3. 3	P1	Унификация обработки ошибок	все API routes
3. 4	P1	Синхронизация Zod и Prisma статусов	schemas, models
3. 5	P1	Валидация ИНН/КПП/БИК	schemas, references
4.1	P2	Реализация ролей (Технолог и др.)	UI components
4. 2	P2	Смена пароля пользователем	auth, UI
4. 3	P2	Пагинация и фильтрация	API, UI
4. 4	P2	Дашборд супервизора	stats, UI
4. 5	P2	Подтверждение удаления	UI, soft-delete
4. 6	P2	Загрузка изображений	upload, sharp
5.1	P2	Мобильная адаптация	CSS, layout
5. 2	P2	Уведомления	WebSocket/SSE
5. 3	P2	Экспорт в Excel/CSV	export API
5. 4	P3	Очистка зависимостей	package.json
5. 5	P3	Удаление legacy Task	schema, routes
6.1	P3	Оптимизация N+1 запросов	API routes
6. 2	P3	Подготовка к миграции на PostgreSQL	database
6. 3	P3	Добавление индексов	schema.prisma

8. Рекомендуемый порядок реализации

Спринт 1 — Безопасность (1-2 недели)

- 2.1 Починка JWT-верификации
- 2.2 Защита seed и print
- 2.3 Починка /api/auth/session
- 2.4 Добавление транзакций в критические операции
- 3.5 Валидация ИНН/КПП/БИК

Спринт 2 — Качество (1-2 недели)

- 3.1 Устранение TypeScript-антипаттернов
- 3.2 Настройка Vitest + тесты auth
- 3.3 Унификация обработки ошибок
- 3.4 Синхронизация Zod и Prisma
- 5.4 Очистка зависимостей
- 5.5 Удаление legacy Task

Спринт 3 — Функциональность (2-3 недели)

- 4.2 Смена пароля
- 4.5 Подтверждение удаления + soft-delete
- 4.3 Пагинация и фильтрация
- 4.4 Дашборд супервизора
- 4.6 Загрузка изображений

Спринт 4 — Роли и UX (2-3 недели)

- 4.1 Реализация ролей (Раскройщик, Технолог, Продавец)
- 5.1 Мобильная адаптация
- 5.2 Уведомления
- 5.3 Экспорт данных

Спринт 5 — Оптимизация (1-2 недели)

- 6.1 Оптимизация N+1 запросов
- 6.2 WAL-режим SQLite + подготовка к PostgreSQL

· 6.3 Добавление индексов